Construire des graphiques avec Python

Lino Galiana
Version du 20 octobre 2026

Une partie essentielle du travail du *data scientist* est d'être en mesure de synthétiser une information dans des représentations graphiques percutantes. Ce chapitre permet de découvrir les enjeux de la représentation de données avec `Python`, l'écosystème pour faire ceci. Il ouvre également à la représentation interactive de données avec `Plotly` ou `Altair`.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **2**
Données 3
2 Premières productions graphiques avec l'API `Matplotlib` de `Pandas` **5**
2.1 Comprendre, en quelques mots, le principe de `matplotlib` 5
2.2 Approche explicite (approche orientée POO) 6
2.3 Approche implicite 7
2.4 Découvrir `matplotlib` par l'intermédiaire de `Pandas` 8
3 Utiliser directement `seaborn` **9**
3.1 Comprendre `seaborn` en quelques lignes 9
3.2 Reproduction de l'exemple précédent avec `seaborn` 10
4 Découvrir `Plotnine`, la grammaire des graphiques en `Python` **12**
4.1 La trame de la figure: `ggplot()` 13
4.2 Ajouter les géométries: `geom_*` 13
4.3 Modifier les échelles: `scale_*` 14
4.4 Les labels et les thèmes 14
5 Visualisations alternatives **15**
5.1 Les lollipop chart 16
5.2 Un tableau stylisé 19
6 Des graphiques réactifs grâce aux interfaces avec `Javascript` **22**
6.1 L'écosystème disponible depuis `Python` 22
6.2 La librairie `Plotly` 23
6.3 Réplication de l'exemple précédent avec `Plotly` 23
6.4 La librairie `altair` 24
Références **25**
Informations additionnelles 25
Bibliographie **27**

Compétences à l'issue de ce chapitre

- Découvrir l'écosystème `matplotlib` et `seaborn` pour la construction de graphiques par enrichissement successif de couches.
- Découvrir le récent écosystème `plotnine`, qui est une implémentation en `Python` du *package* `R ggplot2` pour ce type de représentation et qui, grâce à sa grammaire des graphiques, offre une syntaxe puissante pour construire des visualisations de données.

- Découvrir le principe des représentations interactives HTML (format *web*) grâce aux packages `plotly` et `altair`.
- Apprendre les enjeux de la représentation graphique, les compromis nécessaires pour construire un message clair et les limites de certaines représentations classiques.

1 Introduction

Ce chapitre est consacré à la visualisation de données et propose une tâche classique du quotidien des *data scientists* et *data engineers* : la construction de figures qui viendront alimenter un tableau de bord (*dashboard*) analytique permettant d'avoir un regard restrospectif sur un phénomène d'intérêt.

Pour faire ceci, nous allons répliquer quelques figures disponibles en ligne sur le portail *open data* de la ville de Paris . Comme ces figures ne sont pas toujours parfaites du point de vue des bonnes pratiques graphiques, nous allons parfois essayer de rendre l'information représentée plus facilement accessible en modifiant à la marge les figures.

L'objectif principal de ce chapitre est de présenter, rapidement, les principaux écosystèmes de visualisation disponibles en `Python` afin d'en mesurer les forces et faiblesses et connaître, pour un objectif donné, les principaux points d'entrée. Il ne s'agit pas de faire un inventaire complet des graphiques pouvant être fait avec `Python`, ce serait long, assez insipide et peu pertinent car des sites le font déjà très bien à partir d'une grande variété d'exemple, notamment le site python-graph-gallery.com/ . L'objectif est plutôt d'illustrer, par la pratique, quelques enjeux liés à l'utilisation des principales librairies graphiques de `Python`.

> ! Important
>
> Être capable de construire des visualisations de données intéressantes est une compétence nécessaire à tout *data scientist* ou chercheur. Pour améliorer la qualité de ces visualisations, il est recommandé de suivre certains conseils donnés par des spécialistes de la *dataviz* sur la sémiologie graphique.
>
> Les bonnes visualisations de données, comme celles du *New York Times*, reposent certes sur des outils adaptés (des librairies `JavaScript`) mais aussi sur certaines règles de représentation qui permettent de comprendre en quelques secondes le message d'une visualisation.
>
> Transmettre une information synthétique de manière limpide à un public ne s'inventant pas, il est recommandé de réfléchir à la réception d'une visualisation et aux messages principaux que celle-ci est censée transmettre. Cette présentation d'Eric Mauvière illustre, avec de nombreux exemples, la manière dont des choix de visualisation affectent la pertinence du message délivré.
>
> Parmi les autres ressources que j'ai trouvées utiles par le passé, ce post de blog de `datawrapper` (une référence dans le domaine de la visualisation) est très intéressant. Ce post de blog d'Albert Rapp montre également comment construire graduellement une bonne visualisation de données et mérite d'être relu de temps en temps. Enfin, parmi les sites à consulter fréquemment sur le sujet, les ressources proposées sur le blog d'Andrew Heiss valent le détour.

On peut distinguer quelques grandes familles de représentations graphiques: les représentations de distributions propres à une variable, les représentations de relations entre plusieurs variables, les cartes qui permettent de représenter dans l'espace une ou plusieurs variables…

Ces familles se ramifient elles-mêmes en de multiples types de figures. Par exemple, selon la nature du phénomène, les représentations de relations peuvent prendre la forme d'une série temporelle (évolution d'une variable dans le temps), d'un nuage de point (corrélation entre deux variables), d'un

diagramme en barre (pour souligner le rapport relatif entre les valeurs d'une variable en fonction d'une autre), etc.

Plutôt qu'un inventaire à la Prévert des types de visualisations possibles, ce chapitre et le suivant vont plutôt proposer quelques visualisations qui pourraient donner envie d'aller plus loin dans l'analyse avant la mise en oeuvre d'une forme de modélisation. Ce chapitre est consacré aux visualisations traditionnelles, le suivant est dédié à la cartographie. Ces deux chapitres font partie d'un tout visant à offrir les premiers éléments pour synthétiser l'information présente dans un jeu de données.

Le pas suivant est d'approfondir le travail de communication et de synthèse par le biais de productions pouvant prendre des formes aussi diverses que des rapports, des publications scientifiques ou articles, des présentations, une application interactive, un site web ou des *notebooks* comme ceux proposés par ce cours. Le principe général est identique quel que soit le *medium* utilisé et intéresse particulièrement les *data scientists* lorsqu'ils font appel à de l'exploitation intensive de données et désirent obtenir un *output* reproductible. Un jour peut-être j'ajouterai un chapitre à ce propos dans ce cours[1].

> ! Utiliser une interface interactive pour visualiser les graphiques
>
> Pour les chapitres de visualisation, il est vivement recommandé d'utiliser `Python` par le biais d'une interface interactive comme un *notebook Jupyter* (via `VSCode` ou `Jupyter` par exemple, cf. le chapitre de présentation des notebooks).
>
> Cela permet de visualiser les graphiques immédiatement sous chaque cellule de code, de les ajuster facilement, et de tester des modifications en temps réel.
>
> À l'inverse, si l'on exécute des scripts depuis une console classique (par exemple en écrivant dans un fichier `.py` et en exécutant ligne à ligne avec MAJ+,ENTREE dans `VSCode`) les graphiques ne vont pas s'afficher dans une fenêtre *popup*. Cela nécessite de faire des commandes supplémentaires pour les enregistrer, avant d'ouvrir les exports manuellement et pouvoir corriger le cas échéant le code. L'expérience d'apprentissage en devient plus laborieuse.

Données

Ce chapitre s'appuie sur les données de comptage des passages de vélo dans les points de mesure parisiens diffusés sur le site de l'*open data* de la ville de Paris.

L'exploitation de l'historique récent a été grandement facilité par la diffusion des données au format `Parquet`, un format moderne plus pratique que le CSV. Pour en savoir plus sur ce format, vous pouvez consulter les ressources évoquées dans le paragraphe consacré à ce format dans le dernier chapitre de la partie consacrée à la manipulation de données .

```
import os
import requests
from tqdm import tqdm
import pandas as pd
import duckdb

url = "https://minio.lab.sspcloud.fr/lgaliana/data/python-ENSAE/comptage-velo-donnees-compteurs.parquet"
# problem with https://opendata.paris.fr/api/explore/v2.1/catalog/datasets/comptage-velo-donnees-compteurs/exports/parquet?lang=fr&timezone=Europe%2FParis

filename = 'comptage_velo_donnees_compteurs.parquet'
```

[1] Ce chapitre sera construit autour de l'écosystème `Quarto` . En attendant ce chapitre, vous pouvez consulter la documentation exemplaire de cet écosystème et pratiquer, ce sera le meilleur moyen de découvrir.

```
# DOWNLOAD FILE ------------------------------

# Perform the HTTP request and stream the download
response = requests.get(url, stream=True)

if not os.path.exists(filename):
    # Perform the HTTP request and stream the download
    response = requests.get(url, stream=True)

    # Check if the request was successful
    if response.status_code == 200:
        # Get the total size of the file from the headers
        total_size = int(response.headers.get('content-length', 0))

        # Open the file in write-binary mode and use tqdm to show progress
        with open(filename, 'wb') as file, tqdm(
                desc=filename,
                total=total_size,
                unit='B',
                unit_scale=True,
                unit_divisor=1024,
        ) as bar:
            # Write the file in chunks
            for chunk in response.iter_content(chunk_size=1024):
                if chunk:  # filter out keep-alive chunks
                    file.write(chunk)
                    bar.update(len(chunk))
    else:
        print(f"Failed to download the file. Status code: {response.status_code}")
else:
    print(f"The file '{filename}' already exists.")

# READ FILE AND CONVERT TO PANDAS -------------------------

query = """
SELECT id_compteur, nom_compteur, id, sum_counts, date
FROM read_parquet('comptage_velo_donnees_compteurs.parquet')
"""

# READ WITH DUCKDB AND CONVERT TO PANDAS
df = duckdb.sql(query).df()
```

	id_compteur	nom_compteur	id	sum_counts	date
0	100003098-101003098	106 avenue Denfert Rochereau NE-SO	100003098	36	2024-01-01 03:00:00+00:00
1	100003098-101003098	106 avenue Denfert Rochereau NE-SO	100003098	27	2024-01-01 04:00:00+00:00
2	100003098-101003098	106 avenue Denfert Rochereau NE-SO	100003098	10	2024-01-01 06:00:00+00:00

Pour importer les librairies graphiques que nous utiliserons dans ce chapitre, il faut faire

```
import matplotlib.pyplot as plt
import seaborn as sns
from plotnine import *
```

> ⚠ Warning
>
> Importer des librairies sous la forme `from package import *` n'est pas une très bonne pratique.
>
> Néanmoins, pour un *package* comme `plotnine`, dont nous allons utiliser de nombreuses fonctions, ce serait un peu fastidieux d'importer les fonctions au cas par cas. De plus, cela permet de réutiliser presque tels quels les exemples de code de la librairie `R ggplot`, nombreux sur internet avec démonstrations visuelles. `from package import *` est l'équivalent `Python` de la pratique `library(package)` en `R`.

Puisqu'on va régulièrement recréer des variations de la même figure, nous allons créer des variables pour les labels des axes et le titre:

```
title="Les 10 compteurs avec la moyenne horaire la plus élevée"
xaxis="Nom du compteur"
yaxis="Moyenne horaire"
```

2 Premières productions graphiques avec l'API `Matplotlib` de `Pandas`

Chercher à produire une visualisation parfaite du premier coup est illusoire. Il est beaucoup plus réaliste d'améliorer graduellement une représentation graphique afin, petit à petit, de mettre en avant les effets de structure dans un jeu de données.

Nous allons donc commencer par nous représenter la distribution des passages aux principales stations de mesure. Pour cela nous allons produire rapidement un *barplot* puis l'améliorer graduellement.

Dans cette partie, nous allons ainsi reproduire les deux premiers graphiques de la page d'analyse des données : *Les 10 compteurs avec la moyenne horaire la plus élevée* et *Les 10 compteurs ayant comptabilisé le plus de vélos*. Les valeurs chiffrées des graphiques peuvent être différentes de celles de la page en ligne, c'est normal, car nous ne travaillons pas systématiquement sur les données ayant la même fraîcheur que celles en ligne.

2.1 Comprendre, en quelques mots, le principe de `matplotlib`

`matplotlib` date du début des années 2000 et a émergé pour proposer une alternative en `Python` à la création de graphiques sous `Matlab`, un logiciel propriétaire de calcul numérique. `matplotlib` est donc une librairie assez ancienne, antérieure à l'émergence de `Python` dans l'écosystème du traitement de données. Cela s'en ressent sur la logique de construction de `matplotlib` qui n'est pas toujours intuitive lorsqu'on est familier de l'écosystème moderne de la *data science*. Heureusement, il existe de nombreuses librairies qui s'appuient sur `matplotlib` mais qui visent à fournir une syntaxe plus familière aux *data scientists*.

`matplotlib` propose principalement deux niveaux d'abstraction: la figure et les axes. La figure est, en quelque sorte, la « toile » globale qui contient un ou plusieurs axes dans lesquels s'insèreront des graphiques. Selon les cas, il faudra jouer avec les paramètres de figure ou d'axe, ce qui rend très flexible la construction d'un graphique mais peut également être déroutant car on ne sait jamais trop

quel niveau d'abstraction il faut modifier pour mettre à jour sa figure[2]. Comme le montre la Figure 1, tous les éléments d'une figure sont paramétrables.

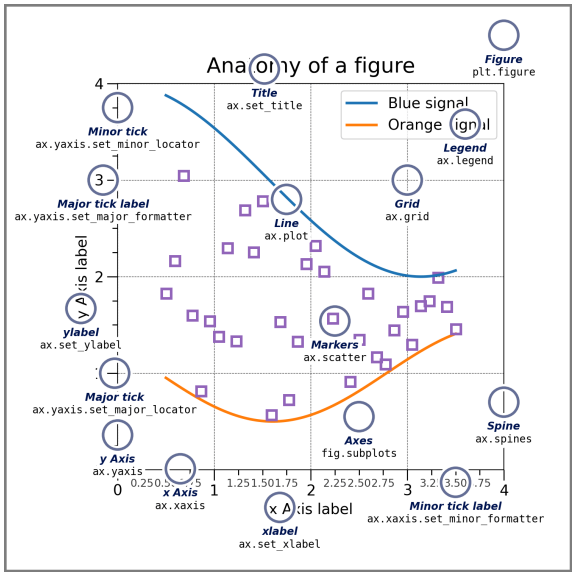


Figure 1. – Comprendre l'architecture d'une figure `matplotlib` (Source: documentation officielle)

En pratique, il existe deux manières de créer et mettre à jour sa figure selon qu'on préfère passer par:

- l'approche explicite, héritière d'une logique de programmation orientée objet, où on crée des objets `Figure` et `Axes` et met à jour ceux-ci.
- l'approche implicite, basée sur l'interface `pyplot` qui utilise une succession de fonctions pour mettre à jour les objets créés implicitement.

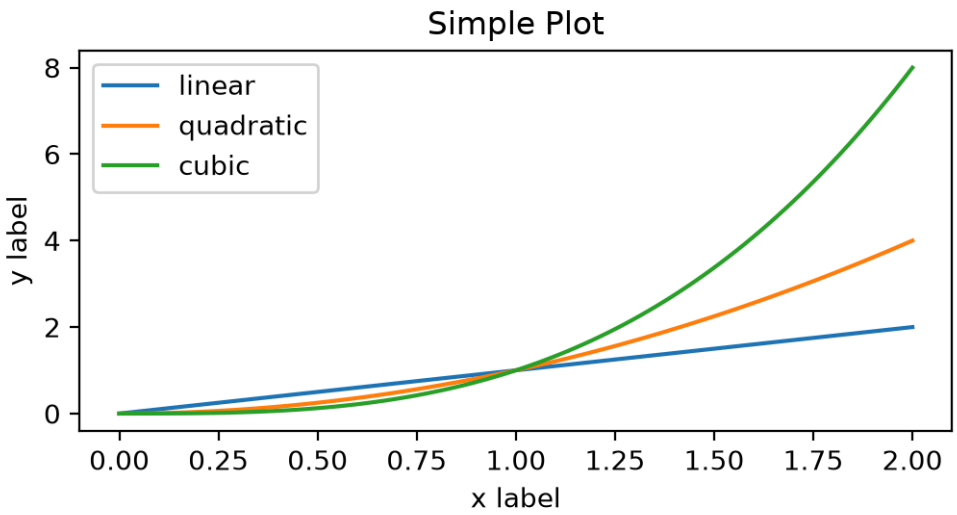
2.2 Approche explicite (approche orientée POO)

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2, 100)  # Sample data.

# Note that even in the OO-style, we use `.pyplot.figure` to create the Figure.
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.plot(x, x, label='linear')  # Plot some data on the Axes.
ax.plot(x, x**2, label='quadratic')  # Plot more data on the Axes...
ax.plot(x, x**3, label='cubic')  # ... and some more.
ax.set_xlabel('x label')  # Add an x-label to the Axes.
ax.set_ylabel('y label')  # Add a y-label to the Axes.
ax.set_title("Simple Plot")  # Add a title to the Axes.
ax.legend()  # Add a legend.
```

[2]Heureusement, comme il existe un énorme corpus de code en ligne utilisant `matplotlib`, les assistants de code comme `ChatGPT` ou `Github Copilot` sont précieux pour construire un graphique à partir d'instructions.



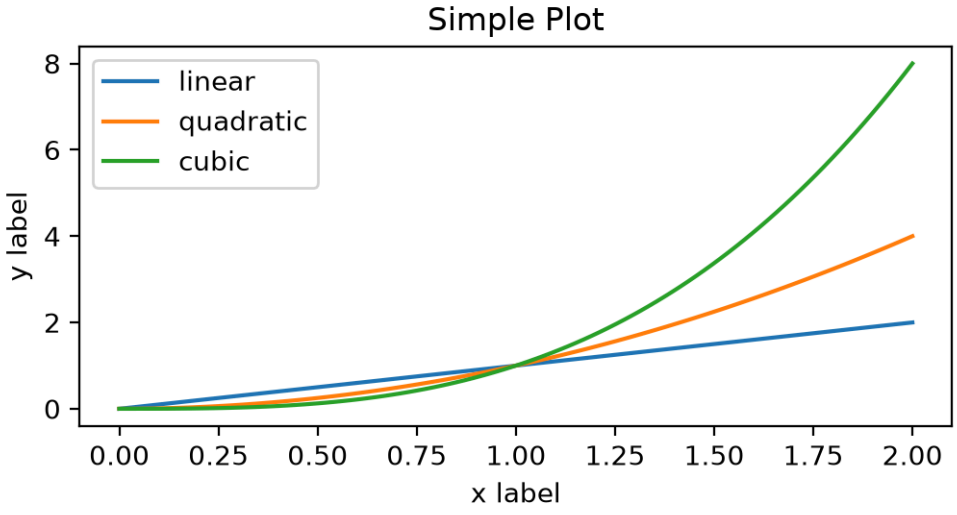
Source: Documentation officielle de `matplotlib`

2.3 Approche implicite

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2, 100)  # Sample data.

plt.figure(figsize=(5, 2.7), layout='constrained')
plt.plot(x, x, label='linear')  # Plot some data on the (implicit) Axes.
plt.plot(x, x**2, label='quadratic')  # etc.
plt.plot(x, x**3, label='cubic')
plt.xlabel('x label')
plt.ylabel('y label')
plt.title("Simple Plot")
plt.legend()
```



Source: Documentation officielle de `matplotlib`

Ces éléments constituent le minimum pour comprendre la logique de `matplotlib`. Pour être plus à l'aise avec ces concepts, la pratique répétée est indispensable.

2.4 Découvrir `matplotlib` par l'intermédiaire de `Pandas`

Il est souvent pratique de produire un graphique rapidement, sans forcément trop se préoccuper du style mais pour avoir une idée, rapide, de la distribution statistique de ses données. Pour cela, l'intégration de fonctionnalités graphiques basiques dans `Pandas` est pratique: on peut directement appliquer quelques instructions sur un `DataFrame` et cela produira une figure `matplotlib`.

L'exercice 1 a pour objet de découvrir ces instructions et la manière dont le résultat peut rapidement être retravaillé pour avoir des statistiques descriptives visuelles.

> Exercice 1 : Produire un premier graphique
>
> Les données comportent plusieurs dimensions pouvant faire l'objet d'une analyse statistique. Nous allons commencer par nous focaliser sur le volume de passage à tel ou tel compteur.
>
> Puisque nous avons comme objectif de synthétiser l'information présente dans notre jeu de données, nous devons d'abord mettre en œuvre quelques agrégations *ad hoc* pour produire un graphique lisible.
>
> 1. Garder les dix bornes à la moyenne la plus élevée. Comme pour obtenir un graphique ordonné du plus grand au plus petit avec les méthodes `plot` de `Pandas`, il faut avoir les données ordonnées du plus petit au plus grand (oui c'est bizarre mais c'est comme ça...), réordonner les données.
> 2. En premier lieu, sans se préoccuper des éléments de style ni de la beauté du graphique, créer la structure du *barplot* (diagramme en bâtons) de la page d'analyse des données.
> 3. Pour préparer le travail sur la deuxième figure, ne conserver que les 10 compteurs ayant comptabilisé le plus de vélos.
> 4. Comme pour la question 2, créer un *barplot* pour reproduire la figure 2 de l'*open data* parisien.

Les 10 principales stations à l'issue de la question 1 représentent celles ayant la moyenne la plus élevée pour le volume de passages de vélos. Ces données réordonnées permettent de créer un graphique lisible et de mettre en avant les stations les plus fréquentées.

	sum_counts
nom_compteur	
72 boulevard Voltaire NO-SE	159.539148
27 quai de la Tournelle SE-NO	166.927660
Quai d'Orsay E-O	178.842743
35 boulevard de Ménilmontant NO-SE	180.364565
Totem 64 Rue de Rivoli Totem 64 Rue de Rivoli Vélos E-O	190.852164

La Figure 2 présente les données sous forme de *barplot* basique. Bien qu'elle montre les informations essentielles, elle manque de mise en page esthétique, de couleurs harmonieuses et d'annotations claires, nécessaires pour améliorer la lisibilité et l'impact visuel.

Figure 1 (cliquer ici pour rétracter)

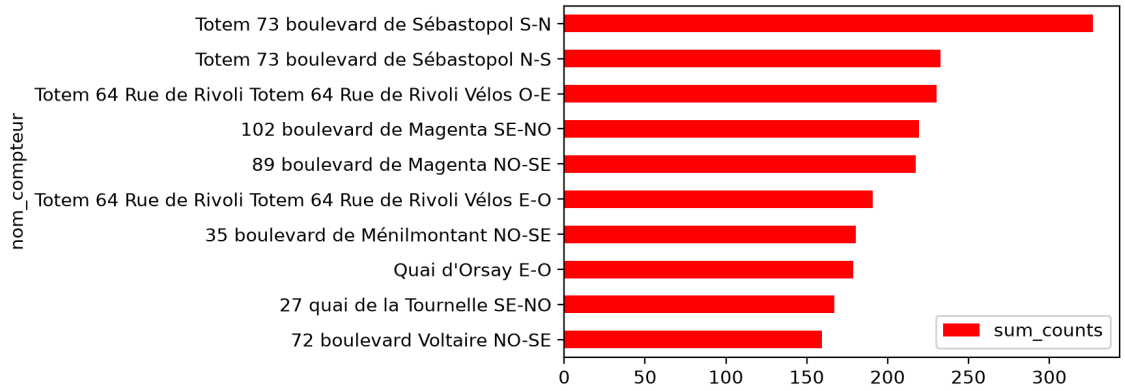


Figure 2. – Première ébauche de la figure « Les 10 compteurs avec la moyenne horaire la plus élevée »

Figure 2 sans travail sur le style (cliquer ici pour rétracter):

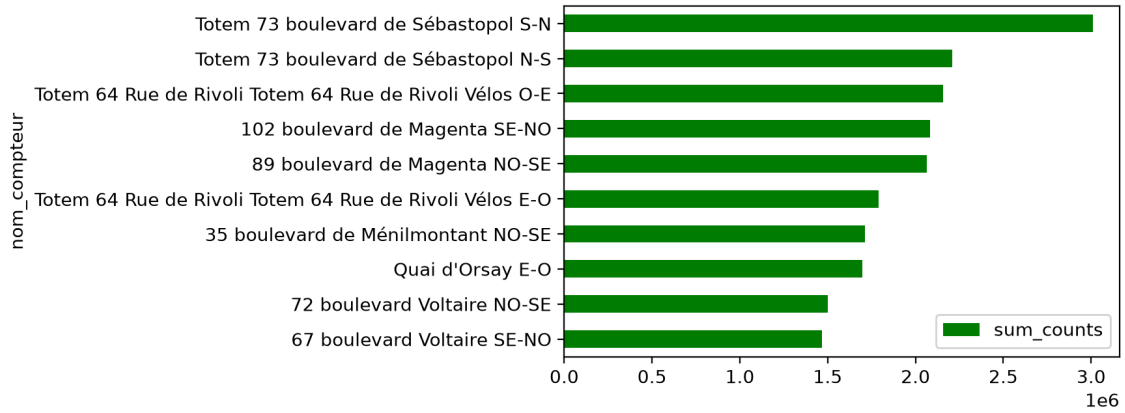


Figure 3. – Première ébauche de la figure « Les 10 compteurs ayant comptabilisés le plus de vélos »

On commence à avoir quelque chose qui transmet un message synthétique sur la nature des données. En l'occurrence, le message qu'on désire faire passer dans cette visualisation est la hiérarchie relative d'usage des stations.

On peut néanmoins remarquer plusieurs éléments problématiques (par exemple les labels) mais aussi des éléments ne correspondant pas (les titres des axes, etc.) ou manquants (le nom du graphique...). La figure est encore brute de décoffrage.

Comme les graphiques produits par `Pandas` suivent la logique très flexible de `matplotlib`, il est possible de les customiser. Cependant, cela demande généralement beaucoup de travail et la grammaire `matplotlib` n'est pas aussi normalisée et intuitive que celle de `ggplot` en `R`. Si on désire rester dans l'écosystème `matplotlib`, il est préférable de directement utiliser `seaborn` , qui offre quelques arguments prêts à l'emploi. Sinon on peut basculer, comme nous le ferons, sur l'écosystème `plotnine` qui offrira la syntaxe normalisée de `ggplot` pour modifier les différents éléments de notre figure.

3 Utiliser directement `seaborn`

3.1 Comprendre `seaborn` en quelques lignes

`seaborn` est une interface haut-niveau au dessus de `matplotlib`. Ce package offre un ensemble de fonctionnalités pour créer des figures ou des axes `matplotlib` directement depuis une fonction admettant de nombreux arguments et, si besoin d'aller plus loin dans la customisation, d'utiliser

les fonctionnalités de `matplotlib` pour mettre à jour la figure, que ce soit par le biais de l'approche implicite ou explicite décrites précédemment.

Comme pour `matplotlib`, `seaborn` permet de faire la même figure de multiples manières. `seaborn` hérite de la dualité axes-figures de `matplotlib` et il faudra souvent jouer avec un niveau ou l'autre. La principale caractéristique de `seaborn` est d'offrir quelques points d'entrée standardisés, par exemple `seaborn.relplot` ou `seaborn.catplot`, et une logique d'*inputs* basée sur le `DataFrame` là où `matplotlib` est structurée autour du *array* `Numpy`. Il faut néanmoins être conscient que `seaborn` souffre globalement des mêmes limites que `matplotlib`, notamment du caractère peu intuitif des éléments de customisation qui, dès lors qu'on ne les trouvent pas dans les arguments, peuvent devenir un casse-tête à mettre en oeuvre.

La figure comporte maintenant un message mais il est encore peu lisible. Il y a plusieurs manières de faire un *barplot* en `seaborn`. Les deux principales sont :

- `sns.catplot` ;
- `sns.barplot`.

On propose d'utiliser `sns.catplot` pour cet exercice. Il s'agit d'un point d'entrée assez fréquent pour faire des graphiques d'une variable discrétisée.

3.2 Reproduction de l'exemple précédent avec `seaborn`

Nous allons nous contenter de reproduire la Figure 2 avec `seaborn`. Pour cela, voici le code nécessaire afin d'avoir un `DataFrame` prêt à l'emploi:

```
df1 = (
    df
    .groupby('nom_compteur')
    .agg({'sum_counts': "mean"})
    .sort_values('sum_counts', ascending = False)
    .head(10)
    .sort_values('sum_counts')
)


df1 = df1.reset_index().sort_values("sum_counts", ascending = False)

df1.head()
```

	nom_compteur	sum_counts
9	Totem 73 boulevard de Sébastopol S-N	327.091037
8	Totem 73 boulevard de Sébastopol N-S	232.560270
7	Totem 64 Rue de Rivoli Totem 64 Rue de Rivoli ...	230.304195
6	102 boulevard de Magenta SE-NO	219.405306
5	89 boulevard de Magenta NO-SE	217.406990

Exercice 2: reproduire la première figure avec seaborn

1. Refaire le graphique précédent avec la fonction `catplot` de `seaborn`. Pour contrôler la taille du graphique vous pouvez utiliser les arguments `height` et `aspect`.
2. Ajouter les titres des axes et le titre du graphique

3. Même si cela n'apporte rien en termes d'information, essayez de colorer en rouge, comme sur la figure du portail *open data*, l'axe des `x`. Vous pouvez pré-définir un style avec `sns.set_style("ticks", {"xtick.color": "red"})`

À l'issue de la question 1, c'est-à-dire en utilisant `seaborn` pour reproduire de manière minimale un *barplot*, on obtient la Figure 4. C'est déjà un peu plus propre que la version précédente (Figure 2) et cela peut déjà suffire pour un travail exploratoire.

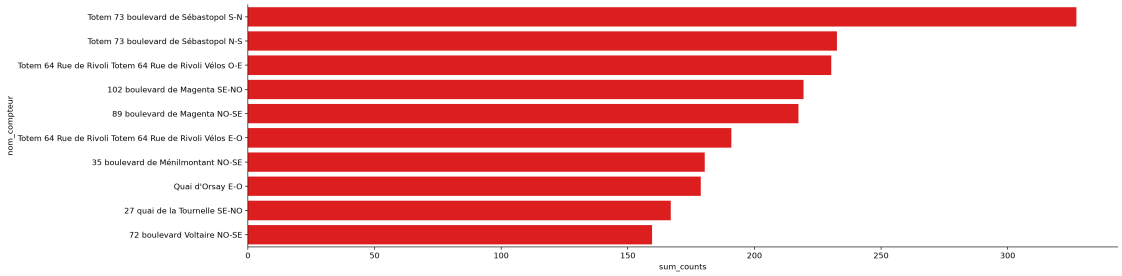


Figure 4

A la fin de l'exercice, on obtient une figure proche de celle qu'on essaie de reproduire. La principale différence tient à l'absence, sur la nôtre, des valeurs numériques.

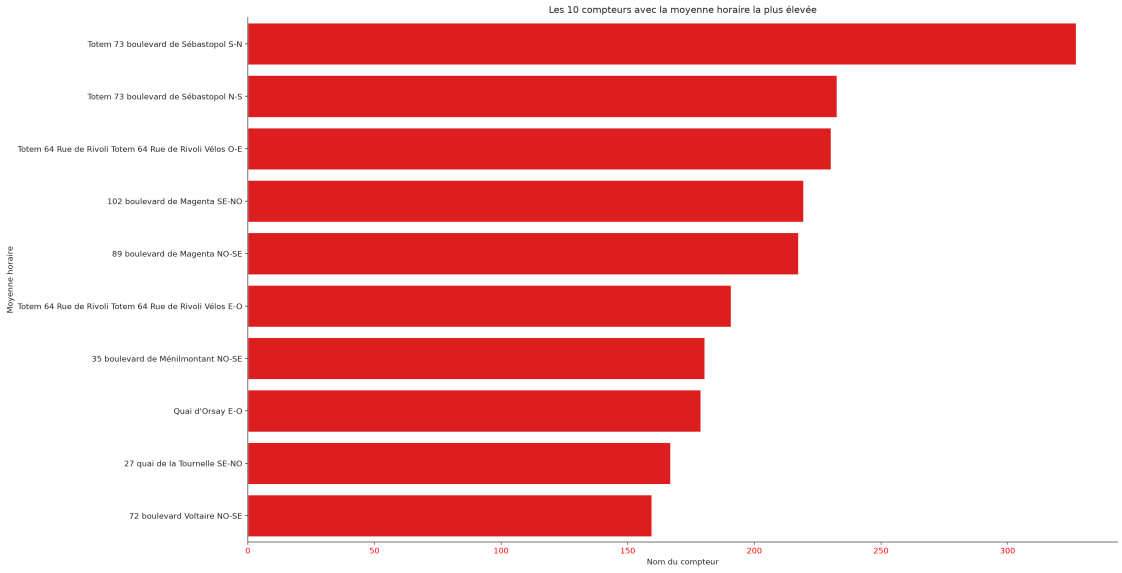


Figure 5

On comprend ainsi que le boulevard de Sébastopol est le plus emprunté, ce qui ne vous surprendra pas si vous faites du vélo à Paris. Néanmoins, si vous n'êtes pas familiers avec la géographie parisienne, cela sera peu informatif pour vous, vous allez avoir besoin d'une représentation graphique supplémentaire : une carte ! Nous verrons ceci lors d'un prochain chapitre.

4 Découvrir `Plotnine`, la grammaire des graphiques en `Python`

`plotnine` est le nouveau venu dans l'écosystème de la visualisation en `Python`. Cette librairie est développée par `Posit`, l'entreprise à l'origine de l'éditeur `RStudio` et de l'écosystème du *tidyverse* si central dans le langage `R`. Cette librairie vise à importer la logique de `ggplot` en `Python`, c'est-à-dire une grammaire des graphiques normalisée, lisible et flexible héritée de L. Wilkinson [1].

Dans cette approche, un graphique est vu comme une succession de couches qui, une fois superposées, donneront la figure suivante. En soi, ce principe n'est pas différent de celui de `matplotlib`. Néanmoins, la grammaire utilisée par `plotnine` est beaucoup plus intuitive et normalisée, ce qui offrira beaucoup plus d'autonomie pour modifier sa figure.



Figure 6. – La logique de `ggplot` (et `plotnine`) image empruntée à Andrew Heiss

Avec `plotnine`, il n'y a plus de point d'entrée dual figure-axe. Comme l'illustrent les slides ci-dessous :

1. On initialise une figure
2. On met à jour les couches (*layers*), un niveau d'abstraction très général concernant aussi bien les données représentées que les échelles des axes ou la couleur
3. À la fin, on peut jouer sur l'esthétique en modifiant les labels des axes, de la légende, les titres, etc.

Nous allons avoir besoin de données hiérarchisées pour avoir des barres ordonnées de manière cohérente:

```
df1["nom_compteur"] = pd.Categorical(
    df1["nom_compteur"],
    categories = df1["nom_compteur"][::-1],
    ordered=True
)
```

Exercice 4: reproduire la première figure avec plotnine

Ceci est le même exercice que l'exercice 2. L'objectif est de faire cette figure avec `plotnine`

Pour cet exercice, nous proposons une correction guidée pas à pas permettant d'illustrer la logique de la grammaire des graphiques.

4.1 La trame de la figure: `ggplot()`

La première étape de toute figure consiste à définir l'objet du graphique, c'est-à-dire les données qui seront visuellement représentées. Cela passe par la déclaration `ggplot` avec les paramètres suivants:

- Le `DataFrame`, premier paramètre de tout appel à `ggplot`.
- Les principaux paramètres esthétiques variables - qui s'insè communs aux différentes couches. En l'occurrence nous n'av pourrions, selon la nature du graphique, avoir d'autres esht contrôlé par une variable de notre jeu de données : couleur, transparence, etc.

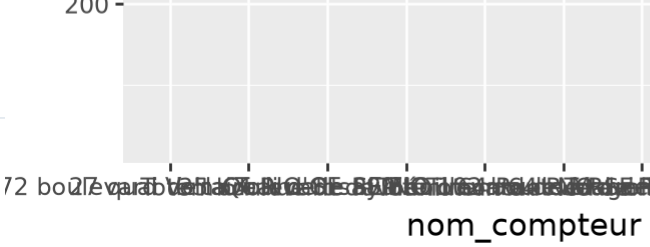


Figure 7. – La trame de base

```
ggplot(df1, aes(x="nom_compteur", y="sum_counts"))
```

Cela nous donne la structure du graphique dans laquelle vont s Concernant les x et y choisis, cette déclaration permettra de déf vertical. Nous verrons ensuite que nous allons renverser les axe viendra plus tard.

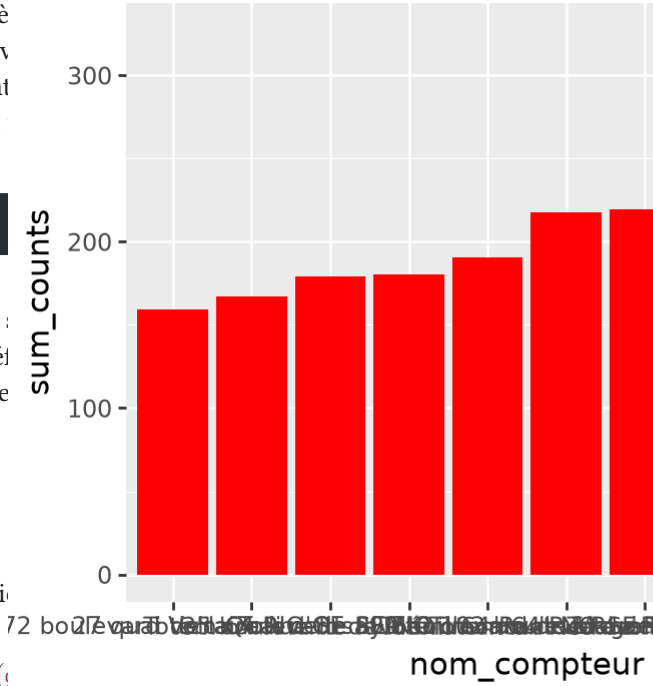


Figure 8. – La première couche de géométrie (bar)

4.2 Ajouter les géométries: `geom_*`

Les couches graphiques sont définies par la famille des fonctio (d'où le `+`). Le contrôle de celles-ci se fait à deux niveaux:

- Dans les paramètres définis par `aes` soit au niveau global (géométrie en question (dans l'appel à `geom_`)
- Dans les paramètres constants qui s'appliquent uniformément à la couche, définis comme des paramètres constants

```
(
    ggplot(df1, aes(x="nom_compteur", y="sum_counts")) +
    geom_bar(stat="identity", fill="red")
)
```

On peut ajouter plusieurs couches successives. Par exemple, le donner du contexte peuvent être créées à partir de `geom_text` do géré par les mêmes paramètres que les autres couches:

```
df1["text"] = df1["sum_counts"].round().astype(int).astype(st

(
    ggplot(df1, aes(x="nom_compteur", y="sum_counts")) +
    geom_bar(stat="identity", fill="red") +
    geom_text(aes(label = "text"), position=position_nudge(y=
)
```

Ligne 6

Ce paramètre de `position` est inutile, voire gênant, pour le n rement à décaler le label (cf. documentation de `plotnine`) qu

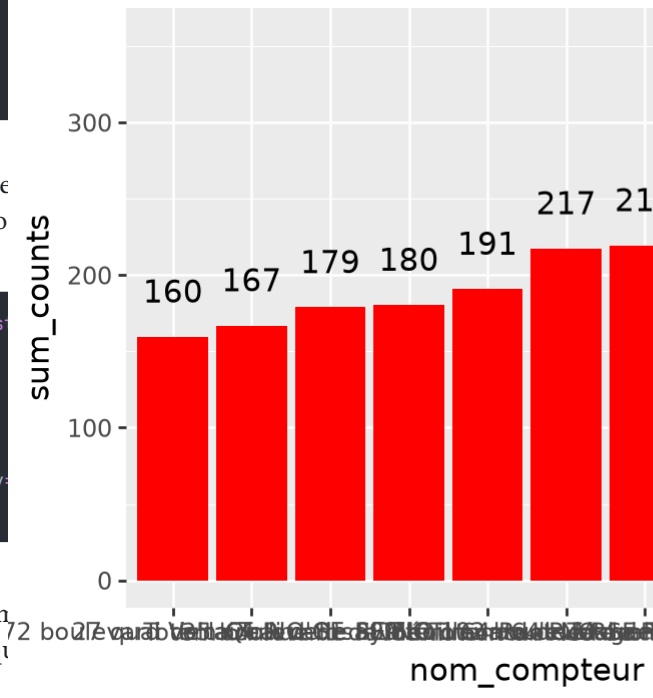


Figure 9. – La seconde couche de géométrie (texte)

4.3 Modifier les échelles: `scale_*`

L'harmonisation des déclarations d'éléments visuels permise pa
avec les géométries `geom_*`. Il est donc logique que le contrôle
de manière uniformisée, par le biais d'une autre famille de fo
`scale_x_continuous`, `scale_color_discrete`...).

Ainsi, chaque esthétique (`x`, `y`, `color`, `fill`, `size`, etc.) peut être fin
matique via sa propre échelle (`scale_*`). Ceci offre un contrôle
des données.

On peut aussi ranger dans cette catégorie les fonctions de la fam
de coordonnées. En l'occurrence, on va se servir de `coord_flip`
verticales.

```
(
    ggplot(df1, aes(x="nom_compteur", y="sum_counts")) +
    geom_bar(stat="identity", fill="red") +
    geom_text(aes(label = "text"), position=position_nudge(y=
    scale_y_continuous(expand=(0, 40)) +
    coord_flip()
)
```

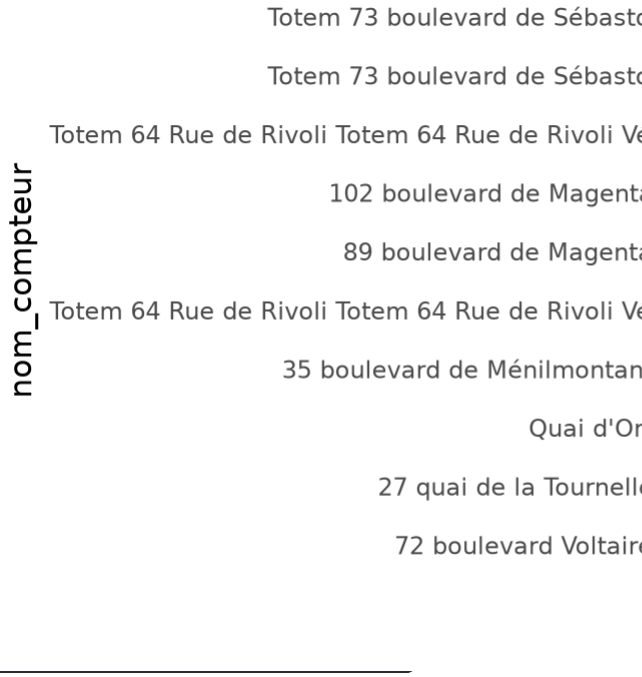


Figure 10. – L'échelle de notre graphique

Ici on a peu de paramètres à modifier puisque nos échelles nous conviennent déjà bien (on a pas à utiliser le *log* pour compresser l'échelle, à appliquer une palette de couleurs...). On va juste aggrandir un peu l'axe des x pour pouvoir rentrer nos valeurs numériques. Comme avant d'échanger les coordonnées avec `coord_flip` l'axe en question est y, on va donc jouer avec `scale_y_continuous`.

4.4 Les labels et les thèmes

La fin de la déclaration de notre figure se fait à travers les éléments de forme que sont les labels (axes, titres, notes de lecture...) et le thème (préconfiguré à travers la famille `theme_` ou personnalisé avec les paramètres de la fonction `theme`). Avant cela, réduisons la taille de nos labels de y

```
import textwrap

def wrap_label(s, width=30):
    return '\n'.join(textwrap.wrap(s, width=width))

df1["nom_compteur"] = df1["nom_compteur"].apply(wrap_label)
```

On peut maintenant personnaliser notre figure:

```
                            p = (
        ggplot(df1, aes(x="nom_compteur", y="sum_counts")) +
              geom_bar(stat="identity", fill="red") +
    geom_text(aes(label = "text"), position=position_nudge(y=30)) +
                scale_y_continuous(expand=(0, 40)) +
                          coord_flip() +
                              labs(
                            title=title,
                             x=xaxis,
                             y=yaxis
                             ) +
                            theme(
            panel_background=element_rect(fill="white"),
                 line=element_line(color="white"),
          axis_text_x=element_text(angle=45, hjust=1, color="red"),
                  axis_title_x=element_text(color="red"),
                               )
                            )

                            p
```

Figure 11

Bien que rapide, cette plongée dans l'univers de la grammaire des graphiques de ggplot montre à quel point celle-ci est intuitive - lorsqu'on comprend sa logique - et puissante.

> **Caution**
>
> Pour contextualiser les données temporelles, on utilise généralement des dates sur l'axe des abscisses (x). Pour éviter de rendre celles-ci illisibles, il faut éviter de donner trop de détail (trop de valeurs affichées, affichage des jours quand les mois suffisent...).
>
> Tourner le texte verticalement pour faire rentrer plus de texte sur l'axe horizontal n'est pas une bonne idée: l'effet principal de cela est de donner un torticolis au lecteur de votre graphique. Il vaut mieux afficher moins de labels, quitte à faire une note de lecture pour les dates vraiment spéciales.

5 Visualisations alternatives

Jusqu'à présent, nous avons consciencieusement reproduit les visualisations proposées sur le tableau de bord de l'*open data* parisien. Mais nous pourrions vouloir faire passer la même information avec des visualisations différentes:

- Les *lollipop chart* ressemblent beaucoup aux diagrammes en barre mais l'information visuelle est un peu plus efficiente: on n'a pas une grosse barre pour représenter les grandeurs mais une ligne plus fine ce qui peut aider à vraiment percevoir les échelles de grandeur dans les données.
- Puisque on a besoin de contextualiser la figure avec les valeurs exactes - en attendant de découvrir le monde de l'interactivité - pourquoi ne pas utiliser un tableau et y insérer des graphiques ? Les tableaux ne sont pas un mauvais *medium* de communication, au contraire, s'ils proposent une information visuelle hiérarchisée ils peuvent mettre être très utiles !

5.1 Les lollipop chart

Les diagrammes en bâtons (*barplot*) sont extrêmement communs, sans doute à cause de l'héritage d'Excel où ces graphiques sont faisables en deux clics. Néanmoins, en ce qui concerne le message à transmettre, ils sont loin d'être parfaits. Par exemple, les barres prennent beaucoup d'espace visuel, ce qui peut brouiller le message à transmettre sur le rapport entre les observations.

Sur le plan sémiologique, c'est-à-dire sur le plan de l'efficacité du message à transmettre, les *lollipop charts* sont préférables : ils transmettent la même information mais avec moins de signes visuels pouvant brouiller sa compréhension.

Les *lollipop charts* ne sont pas parfaits non plus mais sont un peu plus efficaces pour transmettre le message. Pour en savoir plus sur les alternatives au *barplot*, la conférence d'Eric Mauvière pour le réseau des *data scientists* de la statistique publique, dont le message principal est *« Désempilez vos figures »*, mérite le détour (disponible sur le site ssphub.netlify.app/).

Avec `plotnine`, il n'est pas trop complexe de créer un *lollipop chart*. Il suffit d'avoir deux géométries:

1. Le manche de la sucette est créé avec un `geom_segment` ;
2. Le bout de la sucette est créé avec un `geom_point`

```
                                    p = (
                ggplot(df1, aes(x="nom_compteur", y="sum_counts")) +
geom_segment(aes(x="nom_compteur", xend="nom_compteur", y=0, yend="sum_counts"), size=1) +
          geom_point(color="white", fill="red", size=6, stroke=1, shape="o") +
                                    coord_flip() +
                                        labs(
                                       title=title,
                                        x=xaxis,
                                        y=yaxis
                                        ) +
                                    theme_minimal()
                                        )

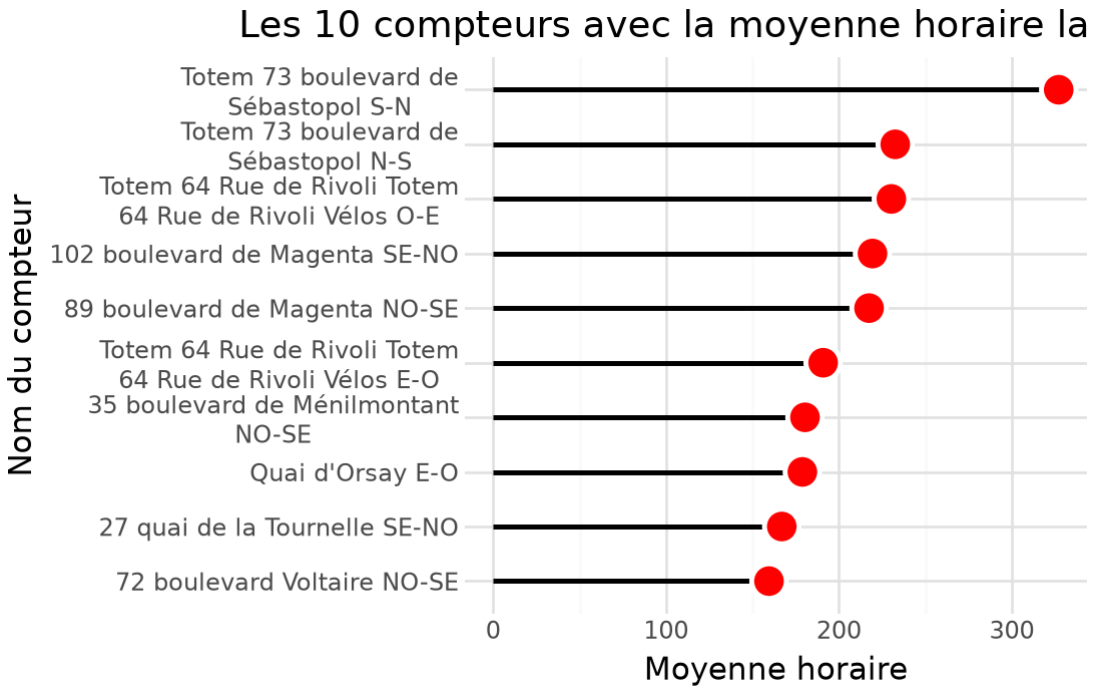
                                        p
```

Figure 12

```
p = (
ggplot(df1, aes(x="nom_compteur", y="sum_counts")) +
geom_segment(
aes(x="nom_compteur", xend="nom_compteur", y=0, yend="sum_counts"), size=1, color
= "white"
) +
geom_point(color="white", fill="red", size=6, stroke=1, shape="o") +
coord_flip() +
labs(
title=title,
x=xaxis,
y=yaxis
) +
theme_minimal() +
theme(
plot_background=element_rect(fill="black"),
panel_background=element_rect(fill="black"),
line=element_line(color="black"),
axis_text_x=element_text(color="white"),
axis_title_x=element_text(color="white"),
text=element_text(color="white"),
plot_title=element_text(ha="left")
)
)

p
```

Figure 13



Grâce à cette représentation alternative, on se représente ici mieux la différence entre le compteur le plus utilisé et les autres.

Le *lollipop chart* est une représentation assez classique en biostatistiques ou économie pour représenter les *odds ratio* issus d'une modélisation logistique. En l'occurrence, les lignes servent généralement à représenter la taille de l'intervalle de confiance dans cette littérature.

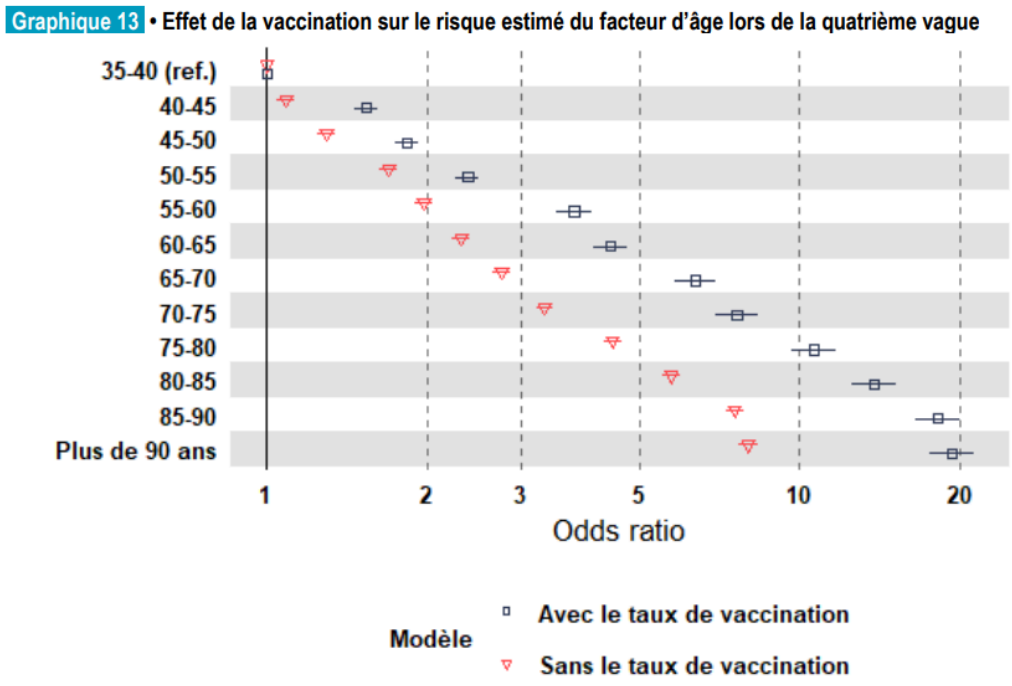


Figure 14. – Une variante des *lollipop charts* pour représenter les *odds ratio* [2]

Une variante des *lollipop chart*, popularisée notamment par *datawrapper* permet aussi de représenter des intervalles: le *range plot*. Il permet à la fois de se représenter la hiérarchie entre des observations et l'amplitude d'un phénomène.

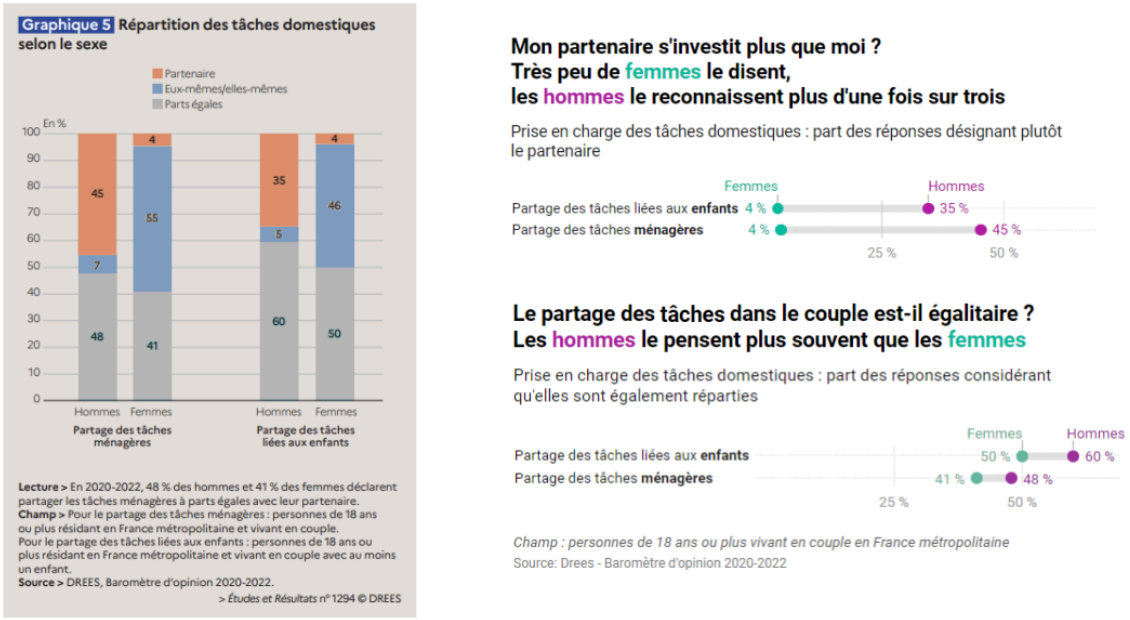


Figure 15. – Exemple de *range plot* par Eric Mauvière (ssphub.netlify.app/).

5.2 Un tableau stylisé

Les tableaux sont de bons *medium* pour communiquer des valeurs précises. Mais sans l'ajout d'éléments contextuels, comme des intensités de couleurs ou des figures, ils aident peu à percevoir visuellement les écrats ou les ordres de grandeur.

Grâce à la richesse du format HTML - qui autorise l'insertion de graphismes légers directement dans les cellules - on peut combiner précision numérique et lisibilité visuelle. On a donc la possibilité d'avoir le meilleur des deux mondes.

Nous avons déjà utilisé précédemment le package `great_tables` pour représenter des statistiques agrégées. Nous allons ici l'utiliser pour intégrer un *lollipop chart* dans un tableau, permettant une lecture immédiate des valeurs tout en conservant leur exactitude.

Nous allons en profiter pour nettoyer un peu le texte à afficher en retirant les *labels* doublonnés et en isolant la direction.

```
df1["direction"] = df1["nom_compteur"].str.extract(
    r"([A-Z]{1,3}-[A-Z]{1,3})$"
)
df1["nom_compteur"] = df1["nom_compteur"].str.replace(
    r"([A-Z]{1,3}-[A-Z]{1,3})$", "", regex=True
)

def deduplicate_label(label):
    parts = label.split()
    mid = len(parts) // 2
    for i in range(1, mid + 1):
        if parts[:i] == parts[i:2*i]:
            return ' '.join(parts[i:])
    return label

df1["nom_compteur"] = df1["nom_compteur"].apply(deduplicate_label)
df1["nom_compteur"] = df1['nom_compteur'].str.replace("(Vélos|Totem)", "", regex=True)

df1.head()
```

	nom_compteur	sum_counts	text	direction
9	73 boulevard de\nSébastopol	327.091037	327	S-N
8	73 boulevard de\nSébastopol	232.560270	233	N-S
7	64 Rue de Rivoli	230.304195	230	O-E
6	102 boulevard de Magenta	219.405306	219	SE-NO
5	89 boulevard de Magenta	217.406990	217	NO-SE

Nous allons aussi créer une colonne intermédiaire pour créer un visuel synthétique de couleur permettant de voir les compteurs sur plusieurs lignes.

```
import matplotlib.pyplot as plt

df1["nom_compteur_temp"] = df1["nom_compteur"]

# Discrete colormap
categories = df1["nom_compteur_temp"].unique()
cmap = plt.get_cmap("Dark2")

# Create mapping from label to color hex
colors = {cat: cmap(i / max(len(categories) - 1, 1)) for i, cat in enumerate(categories)}
```

```
colors = {k: plt.matplotlib.colors.to_hex(v) for k, v in colors.items()}

# Function to return colored cell
def create_color_cell(label: str) -> str:
    color = colors.get(label, "#ccc")
    return f"""
    <div style="
        width: 20px;
        height: 20px;
        background-color: {color};
        border-radius: 3px;
        margin: auto;
    "></div>
    """
```

Nous allons créer un dictionnaire pour renommer nos colonnes sous une forme plus intelligible que le nom des variables ainsi qu'une variable pour le référencement de la source:

```
columns_mapping = {
    "nom_compteur": "Localisation",
    "direction": "",
    "text": "",
    "sum_counts": "",
    "nom_compteur_temp": ""
}
source_note = "**Source**: Compteurs vélib sur la page de l'[open data de la ville de Paris](https://opendata.paris.fr/explore/dataset/comptage-velo-donnees-compteurs/dataviz/?disjunctive.id_compteur&disjunctive.nom_compteur&disjunctive.id&disjunctive.name)"
```

Le fonctionnement de `great_tables` est assez similaire à celui de `plotnine` : on construit la table par ajout successif de couches, en combinant les fonctions (avec l'opérateur `.` cette fois, et non avec `+`à).

Chaque couche permet d'ajuster un aspect particulier de la table (format des cellules, titres, colonnes, styles, etc.), selon une logique déclarative et cohérente, fidèle à la philosophie de la grammaire des graphiques.

```
from great_tables import *

df_table = df1.loc[:, ["nom_compteur_temp","nom_compteur", "direction", "text", "sum_counts"]]

gt = (
    GT(
        df_table
    )
    .fmt(fns=create_color_cell, columns="nom_compteur_temp")
    .fmt_nanoplot(columns="sum_counts")
    .tab_spanner(
        label="Compteur",
        columns=["nom_compteur_temp", "nom_compteur", "direction"]
    )
    .tab_spanner(
        label="Moyenne horaire",
        columns=["text", "sum_counts"]
    )
    .cols_label(
        **columns_mapping
    )
    .cols_width(
```

```
        cases={
            "nom_compteur_temp": "10%",
            "nom_compteur": "40%",
            "direction": "10%",
            "text": "10%"
        }
    )
    .tab_style(
        style=[
            style.text(size = "small")
        ],
        locations = loc.body(df_table.columns.tolist())
    )
    .tab_style(
        style=[
            style.text(weight="bold")
        ],
        locations=loc.body(columns="text")
    )
    .tab_source_note(
        md(source_note)
    )
)
```

Pour rendre mieux sur un fond noir, on peut ajouter quelques paramètres spécifiques à ce besoin.

```
gt_dark = gt.tab_options(
    table_background_color="black",
    heading_background_color="black"
)
gt_dark
```

Table 1. – Visualisation alternative sous forme de table

Compteur		Moyenne horaire	
Localisation			
73 boulevard de Sébastopol	S-N	**327**	327
73 boulevard de Sébastopol	N-S	**233**	233
64 Rue de Rivoli	O-E	**230**	230
102 boulevard de Magenta	SE-NO	**219**	219
89 boulevard de Magenta	NO-SE	**217**	217
64 Rue de Rivoli	E-O	**191**	191
35 boulevard de Ménilmontant	NO-SE	**180**	180
Quai d'Orsay	E-O	**179**	179
27 quai de la Tournelle	SE-NO	**167**	167
72 boulevard Voltaire	NO-SE	**160**	160

Source: Vélib counters on the Paris open data page

6 Des graphiques réactifs grâce aux interfaces avec `Javascript`

> ! Important
>
> On appelle *tooltip* le texte qui s'affiche en surbrillance lorsqu'on passe la souris sur un élément de la figure (ordinateur) ou lorsqu'on clique sur la figure (smartphone). Il s'agit donc d'un niveau d'information supplémentaire permis par l'interactivité, qui peut être pratique pour alléger le message principal d'une figure.
>
> Néanmoins, comme toute information sur une figure, du travail pour rendre celle-ci utile est nécessaire. On peut rarement se contenter du *tooltip* par défaut d'une librairie: il faut réfléchir au message que celui-ci doit passer, complément textuel à l'information visuelle représentée sur la figure.
>
> Encore une fois, nous n'allons pas creuser ce sujet - c'est l'objet, en soi, d'un cours de *data visualisation* - mais avoir en tête cet enjeu est important pour la réception d'une visualisation.
>
> Par ailleurs, un autre élément que nous mettons totalement de côté ici mais qui est important lorsqu'on rentre dans le monde de la visualisation interactive est celui de la *responsiveness*, c'est-à-dire de la capacité d'une visualisation, et plus généralement d'un site web, à être lisible et fonctionnel sur des tailles d'écran différentes. Gérer des tailles d'écran différentes est compliqué mais est indispensable si on vise un public large dont une partie croissante utilise des smartphones pour consulter un site web.
>
> A ceci s'ajoute des questions d'adaptabilité du site à différentes formes de handicaps visuels. Par exemple, ce serait environ 8% des hommes qui seraient atteints d'une forme de daltonisme, avec des formes différentes (6% ont des difficultés à percevoir les nuances de vert, 2% de rouge).
>
> Bref, vous qui entrez dans le monde de la visualisation, laissez toute espérance de simplicité. Les outils sont simples d'usage mais les besoins sont complexes.

6.1 L'écosystème disponible depuis `Python`

Les figures construites avec `matplotlib` ou `plotnine` sont figées et présentent ainsi l'inconvénient de ne pas permettre d'interaction avec le lecteur. Toute l'information doit donc être contenue dans la figure, ce qui peut la rendre difficile à lire. Si la figure est bien faite, avec différents niveaux d'information, cela peut bien fonctionner.

Il est néanmoins plus simple, grâce aux technologies *web*, de proposer des visualisations à plusieurs niveaux. Un premier niveau d'information, celui du coup d'œil, peut suffire à assimiler les principaux messages de la visualisation. Ensuite, un comportement plus volontaire de recherche d'information secondaire peut permettre d'en savoir plus. Les visualisations réactives, qui sont maintenant la norme dans le monde de la *dataviz*, permettent ce type d'approche : le lecteur d'une visualisation peut passer sa souris à la recherche d'informations complémentaires (par exemple, les valeurs exactes) ou cliquer pour faire apparaître des informations complémentaires sur la visualisation ou autour.

Ces visualisations reposent sur le même triptyque que l'ensemble de l'écosystème *web* : `HTML`, `CSS` et `JavaScript`. Les utilisateurs de `Python` ne vont jamais manipuler directement ces langages, qui demandent une certaine expertise, mais vont utiliser des librairies au niveau de `R` qui généreront automatiquement tout le code `HTML`, `CSS` et `JavaScript` permettant de créer la figure.

Il existe plusieurs écosystèmes `Javascript` mis à disposition des développeurs.euses par le biais de `Python`. Les deux principales librairies sont `Plotly`, associée à l'écosystème `Javascript` du même nom,

et `Altair`, associée à l'écosystème `Vega` et `Altair` en `Javascript`[3]. Pour permettre aux pythonistes de découvrir la librairie `Javascript` émergente `Observable Plot`, l'ingénieur de recherche français Julien Barnier a développé `pyobsplot` une librairie `Python` permettant d'utiliser cet écosystème depuis `Python`.

L'interactivité ne doit pas juste être un gadget n'apportant pas de lisibilité supplémentaire, voire la détériorant. Il est rare de pouvoir se contenter de la figure produite sans avoir à fournir un travail supplémentaire pour la rendre efficace.

6.2 La librairie `Plotly`

Le package `Plotly` est une surcouche à la librairie `Javascript Plotly.js` qui permet de créer et manipuler des objets graphiques de manière très flexible afin de produire des objets réactifs sans avoir à recourir à Javascript.

Le point d'entrée recommandé est le module `plotly.express` (documentation ici) qui offre une approche intuitive pour construire des graphiques pouvant être modifiés *a posteriori* si besoin (par exemple pour *customiser* les axes).

> ℹ Visualiser les figures produites par Plotly
>
> Dans un *notebook* `Jupyter` classique, les lignes suivantes de code permettent d'afficher le résultat d'une commande `Plotly` sous un bloc de code :
>
> Pour `JupyterLab`, l'extension `jupyterlab-plotly` s'avère nécessaire:
>
> ```
> !jupyter labextension install jupyterlab-plotly
> ```

6.3 Réplication de l'exemple précédent avec `Plotly`

Les modules suivants seront nécessaires pour construire des graphiques avec `plotly`:

```
import plotly
import plotly.express as px
```

> 💡 Exercice 7: un barplot avec Plotly
>
> L'objectif est de reconstruire le premier diagramme en barre rouge avec `Plotly`.
>
> 1. Réalisez le graphique en utilisant la fonction adéquate avec `plotly.express` et...
>   - Ne pas prendre le thème par défaut mais un à fond blanc, pour avoir un résultat ressemblant à celui proposé sur le site de l'*open-data*.
>   - Pour la couleur rouge, vous pouvez utiliser l'argument `color_discrete_sequence`.
>   - Ne pas oublier de nommer les axes.
> 2. Modifier le texte apparaissant en surbrillance (*hover text*).
> 3. Choisir un fond blanc ou noir pour avoir une couleur de fond uniforme.

[3] Le nom de ces librairies est inspiré de la constellation du triangle d'été dont Véga et Altair sont deux membres.

```
Unable to display output for mime type(s): text/html
```

(a)

```
Unable to display output for mime type(s): text/html
```

(b)

Figure 16. –

6.4 La librairie `altair`

Pour cet exemple, nous allons reconstruire notre figure précédente.

Comme `ggplot`/`plotnine`, `Vega` est un écosystème graphique visant à proposer une implémentation de la grammaire des graphiques de L. Wilkinson [1]. La syntaxe de `Vega` est donc basée sur un principe déclaratif : on déclare une construction par couches et transformations de données progressives.

À l'origine, `Vega` est basée sur une syntaxe JSON, d'où son lien fort avec `Javascript`. Néanmoins, il existe une API Python qui permet de faire ce type de figures interactives nativement en Python. Pour comprendre la logique de construction d'un code `altair`, voici comment répliquer la figure précédente avec :

```
import altair as alt

fig_altair = (
    alt.Chart(df1.reset_index())
    .mark_bar(color='steelblue')
    .encode(
        x=alt.X('sum_counts:Q', title=xaxis),
        y=alt.Y('nom_compteur:N', sort='-x', title=yaxis),
        tooltip=[
            alt.Tooltip('nom_compteur:N', title=xaxis),
            alt.Tooltip('sum_counts:Q', title=yaxis)
        ]
    ).properties(
        title=title
    ).configure_view(
        strokeOpacity=0
    )
)

fig_altair.interactive()
```

```
alt.Chart(...)
```

```
/tmp/ipykernel_340/1789554838.py:2: AltairDeprecationWarning:
Deprecated since `altair=5.5.0`. Use altair.theme instead.
Most cases require only the following change:

    # Deprecated
    alt.themes.enable('quartz')

    # Updated
    alt.theme.enable('quartz')
```

```
If your code registers a theme, make the following change:

    # Deprecated
    def custom_theme():
        return {'height': 400, 'width': 700}
    alt.themes.register('theme_name', custom_theme)
    alt.themes.enable('theme_name')

    # Updated
    @alt.theme.register('theme_name', enable=True)
    def custom_theme():
        return alt.theme.ThemeConfig(
            {'height': 400, 'width': 700}
        )

See the updated User Guide for further details:
    https://altair-viz.github.io/user_guide/api.html#theme
    https://altair-viz.github.io/user_guide/customization.html#chart-themes
```

```
alt.Chart(...)
```

Références

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
```

```
    "gt-extras>=0.0.8",
    "ipykernel>=6.29.5",
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie

[1] L. Wilkinson, « The grammar of graphics », in *Handbook of computational statistics: Concepts and methods*, Springer, 2011, p. 375-414.

[2] L. Galiana, O. Meslin, N. Courtejoie, et S. Delage, « Caractéristiques socio-économiques des individus aux formes sévères de Covid-19 au fil des vagues épidémiques », 2022.